

React useContext - Ausführliche Erklärung mit Beispielen

Warum gibt es useContext?

Ohne useContext müssen Daten oft über viele Komponenten per Props weitergegeben werden. Das nennt man Prop Drilling. Je größer die Anwendung wird, desto unübersichtlicher wird der Code.

Beispiel ohne useContext

```
function App() {
  const user = "Mahmoud";
  return <Layout user={user} />;
}

function Layout({ user }) {
  return <Header user={user} />;
}

function Header({ user }) {
  return <UserInfo user={user} />;
}

function UserInfo({ user }) {
  return <h1>{user}</h1>;
}
```

Problem

Layout und Header benötigen die Daten gar nicht. Sie reichen sie nur weiter. Genau dieses Problem löst useContext.

Schritt 1: Context erstellen

```
import { createContext } from "react";

export const UserContext = createContext("");
```

Erklärung

createContext erstellt einen globalen Container. Der leere String ist der Standardwert.

Schritt 2: Provider erstellen

```
function App() {
  const user = "Mahmoud";

  return (
    <UserContext.Provider value={user}>
      <Layout />
    </UserContext.Provider>
  );
}
```

Erklärung Provider

Der Provider stellt den Wert allen Kind-Komponenten zur Verfügung.

Schritt 3: useContext benutzen

```
import { useContext } from "react";

function UserInfo() {
  const user = useContext(UserContext);
  return <h1>{user}</h1>;
}
```

Was passiert intern?

React sucht nach oben bis zum nächsten Provider.
Dort findet es den Wert und gibt ihn zurück.

Visualisierung

```
Provider (Mahmoud)
|
Layout
|
Header
|
UserInfo --> useContext(UserContext)
```

Praxisbeispiel Dark Mode

```
const ThemeContext = createContext("light");

function App() {
  return (
    <ThemeContext.Provider value="dark">
      <Header />
    </ThemeContext.Provider>
  );
}
```

Theme verwenden

```
function Header() {
  const theme = useContext(ThemeContext);
  return <h1>{theme}</h1>;
}
```

useContext mit useState

```
type ThemeContextType = {
  theme: string;
  toggleTheme: () => void;
};
```

Provider mit State

```
const [theme, setTheme] = useState("light");

function toggleTheme() {
  setTheme(prev =>
    prev === "light" ? "dark" : "light"
  );
}
```

Kompletter Provider

```
<ThemeContext.Provider
  value={{
    theme,
    toggleTheme,
  }}
>
  <Header />
</ThemeContext.Provider>
```

Verwendung in Komponenten

```
const context = useContext(ThemeContext);

<button onClick={context.toggleTheme}>
  Wechseln
</button>
```

Reales Login Beispiel

```
type AuthContextType = {
  user: string;
  login: () => void;
  logout: () => void;
};
```

Wann useContext verwenden?

- Login Daten
- Dark Mode
- Sprache
- Warenkorb
- Globale Einstellungen

Wann nicht?

Wenn Daten nur zwischen Eltern- und Kind-Komponente ausgetauscht werden, reichen Props meistens völlig aus.

Merksatz

```
createContext = Container erstellen
Provider = Daten bereitstellen
useContext = Daten abrufen
```